

Evaluación Final Transversal

Encargo con Defensa Técnica

Docente

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación
DSY1103	Desarrollo FullStack 1		40%

1. Situación Evaluativa 1:

	Ejecución práctica	X	Entrega de encargo	X	Presentación / Defensa Técnica		Prueba escrita (formato quiz)
--	--------------------	---	--------------------	---	--------------------------------	--	-------------------------------

2. Instrucciones generales

Descripción
- El Examen Final Transversal (EFT) corresponde a la evaluación integradora del Proyecto Semestral de Arquitectura de Microservicios, desarrollado durante todo el semestre en equipos de 2 a 3 estudiantes. Su propósito es verificar, de forma práctica e individual, el dominio técnico real alcanzado por cada estudiante en el diseño, construcción, validación, documentación y despliegue de una solución basada en microservicios. - Cada equipo deberá mantener y perfeccionar los microservicios construidos en las evaluaciones anteriores, manteniendo coherencia arquitectónica, cohesión entre servicios, separación de responsabilidades y correcta aplicación de reglas de negocio. Como mínimo, se deben mantener al menos 10 microservicios por equipo. pudiendo ampliarse si la arquitectura lo requiere, siempre que se preserve la coherencia, cohesión y separación funcional entre servicios.

- El encargo deberá ser **entregado en la semana 18**, antes de la sesión de evaluación. La **presentación/defensa técnica se realizará el mismo día designado para la presentación del EFT**, con una duración máxima de **15 minutos por estudiante**. La defensa técnica se organizará **al azar**, por lo que todos los equipos deben estar preparados desde el inicio del período de evaluación.
- Aunque el proyecto se trabaja en **grupos**, la **defensa es individual**. Cada estudiante será evaluado de forma personal, en base a su **desempeño, claridad y dominio del proyecto**. La nota dependerá exclusivamente de lo que cada uno demuestre en la defensa técnica, por lo que es fundamental conocer a fondo todos los aspectos del desarrollo.
- Si durante la defensa un/a estudiante **no puede explicar con claridad el proyecto, no logra ejecutar correctamente la aplicación o no sabe modificar el código de forma funcional**, se asumirá que **no participó activamente en el desarrollo**.
- Esto afectará directamente su calificación, que será asignada **según su propio desempeño, independientemente de que el proyecto esté bien construido** o que su compañero/a lo haya defendido correctamente.
- Informar a los/las estudiantes que, al ser un encargo, deberán realizarlo previamente en su tiempo de trabajo autónomo

Propósito de la evaluación:

El propósito de esta evaluación es verificar la capacidad del equipo y de cada estudiante para validar, documentar, explicar y desplegar una arquitectura distribuida basada en microservicios independientes, con comunicación REST, pruebas unitarias, documentación formal y despliegue real, demostrando dominio en:

- **Arquitectura, diseño y organización del proyecto**
 - Aplicación correcta del patrón CSR (Controller–Service–Repository/Model).
 - Diseño coherente de la arquitectura de microservicios, manteniendo cohesión, separación funcional y trazabilidad.
 - Gestión adecuada de propiedades, perfiles y configuración por medio de archivos `application.yml` / `application.properties`.
 - Implementación de buenas prácticas de organización, mantenibilidad y legibilidad del código.
- **Persistencia, reglas de negocio y manejo de datos**
 - Manipulación de bases de datos reales, incluyendo consultas, relaciones y operaciones CRUD completas.
 - Implementación clara de reglas de negocio, validaciones sólidas y manejo centralizado de errores.
 - Integración de lógica aplicada a entidades específicas según el dominio del proyecto.
- **Comunicación e interoperabilidad entre microservicios**

- Construcción de endpoints REST funcionales y eficientes.
- Integración de comunicación distribuida entre microservicios, garantizando interoperabilidad, consistencia y manejo adecuado de fallas.
- Implementación de un API Gateway como punto de entrada unificado para la arquitectura.
- **Validación del sistema mediante pruebas unitarias**
 - Desarrollo de pruebas unitarias para validar reglas de negocio.
 - Explicación del funcionamiento de pruebas, mocks, asserts, casos evaluados y cobertura lógica.
 - Capacidad para implementar nuevas pruebas o corregirlas en tiempo acotado durante la defensa.
- **Documentación y trazabilidad técnica**
 - Elaboración de documentación formal del API utilizando Swagger/OpenAPI.
 - Explicación de la documentación, rutas, recursos y decisiones técnicas del proyecto.
- **Despliegue y operatividad**
 - Configuración y despliegue exitoso de los microservicios en entornos locales o remotos (Docker, Railway, Render u otros).
 - Capacidad para ejecutar el sistema completo, evidenciar funcionalidad y resolver problemas comunes en entorno productivo o semi-productivo.
- **Trabajo colaborativo y control de versiones**
 - Uso adecuado de GitHub para versionamiento, ramas, commits y organización del código.
 - Uso de herramientas colaborativas como Trello para planificación, roles, seguimiento y entrega de evidencia.
 - Trazabilidad clara del aporte individual dentro del trabajo grupal.

Instrucciones específicas de la Evaluación:

Consideraciones para la realización de la ejecución práctica:

Se recomienda realizar esta evaluación en laboratorio o sala equipada con:

- Computadores con acceso a IntelliJ IDEA o VS Code, JDK y todas las dependencias necesarias para Spring Boot.

- Posibilidad de ejecutar Docker Desktop o herramientas equivalentes cuando el despliegue se realice mediante contenedores.
- Accesos configurados a plataformas de despliegue remoto como Railway, Render u otras (cuando se utilicen).
- Conexión a internet estable para:
 - Descarga de dependencias Maven/Gradle.
 - Acceso a repositorios GitHub.
 - Acceso a paneles de despliegue remoto (Railway, Render, etc.).
- Herramientas de prueba como Postman u otro cliente REST.
- Posibilidad de compartir pantalla para que el docente pueda visualizar:
 - Código fuente.
 - Ejecución de pruebas unitarias.
 - Documentación Swagger/OpenAPI.
 - Configuración del Gateway y archivos YAML.
 - Despliegue local y/o remoto en funcionamiento.

Requisitos obligatorios de la evaluación:

El desarrollo debe integrar todos los aprendizajes adquiridos durante las Experiencias de Aprendizaje 2 y 3, incluyendo los siguientes elementos:

- **Persistencia real con JPA + Hibernate**
 - Implementación de entidades con anotaciones como @Entity, @Id, @GeneratedValue, @OneToMany, @ManyToOne, etc.
 - Uso de repositorios mediante JpaRepository para ejecutar operaciones CRUD reales.
 - Configuración del datasource y dialecto en application.properties.
 - Creación y/o ejecución de migraciones iniciales de base de datos (scripts SQL o Flyway/Liquibase).
- **Modelado de datos y normalización básica**
 - Diseño de modelos relacionales coherentes con el dominio del proyecto.
 - Definición de relaciones correctas entre tablas y entidades.
 - Manejo de claves primarias, claves foráneas e integridad referencial.

- **Validaciones con Bean Validation (JSR 380)**
 - Uso de anotaciones
 - Validación de datos en controladores, retornando respuestas consistentes ante entradas inválidas.
 - Separación entre DTOs y entidades para validar datos de manera limpia y segura.
- **Manejo de excepciones básicas y respuestas controladas**
 - Uso de try/catch cuando corresponda en la capa de servicio.
 - Retorno de respuestas coherentes con ResponseEntity, utilizando códigos HTTP adecuados
 - Uso de @ControllerAdvice para manejo centralizado de errores.
- **Estructuración del código bajo el patrón CSR**
 - Separación clara entre:
 - Controller → manejo de solicitudes REST
 - Service → lógica de negocio
 - Repository → acceso a datos
 - Flujo ordenado, coherente y sin mezclar responsabilidades.
 - Uso de DTOs para comunicación entre capas cuando sea apropiado.
- **Respuestas REST correctas y consistentes**
 - Todos los endpoints deben retornar JSON estructurado.
 - Los microservicios deben aplicar buenos principios REST:
 - rutas semánticas
 - métodos HTTP adecuados (GET, POST, PUT, DELETE)
 - parámetros, path variables y request bodies organizados
 - Uso obligatorio de ResponseEntity para controlar la respuesta.
- **Integración de logs estructurados con SLF4J**
 - Mensajes claros que permitan trazabilidad entre capas.
 - Reflejo de eventos relevantes como creación, actualización, errores y validaciones fallidas.
- **Comunicación entre microservicios**
 - Consumo de endpoints entre microservicios mediante:

- WebClient
 - o Feign Client
- Manejo de timeouts, errores y validación de datos recibidos.
- Pruebas de integración mínimas mediante Postman u otra herramienta REST.
- **Pruebas unitarias con JUnit (y Mockito u otro framework de mocking)**
 - Creación de clases de test en src/test/java siguiendo una estructura organizada.
 - Implementación de pruebas unitarias sobre servicios y/o lógica de negocio, validando reglas clave del dominio.
 - Uso de asserts adecuados y estructura Given–When–Then u otra convención clara.
 - Empleo de mocks o dobles de prueba para simular repositorios y dependencias cuando corresponda.
 - Ejecución de las pruebas antes de la defensa, asegurando que se encuentren en estado exitoso.
- **Documentación técnica con Swagger/OpenAPI**
 - Configuración de dependencias necesarias (por ejemplo, springdoc-openapi).
 - Habilitación de la UI de Swagger para la exploración de endpoints.
 - Documentación de los endpoints principales, incluyendo:
 - Descripción funcional.
 - Parámetros de entrada.
 - Códigos de respuesta.
 - Ejemplos de respuesta en JSON.
 - Coherencia entre lo documentado y el comportamiento real del microservicio.
- **Comunicación entre microservicios y API Gateway**
 - Consumo de endpoints entre microservicios mediante WebClient o Feign Client.
 - Manejo de respuestas, validación de datos y tratamiento de errores remotos.
 - Configuración de un API Gateway (Spring Cloud Gateway o equivalente) para centralizar rutas hacia los microservicios.
 - Definición de rutas, prefijos y filtros básicos que permitan un flujo controlado de las solicitudes.
- **Configuración con archivos YAML**
 - Uso de archivos .yml o .yaml para definir propiedades de entorno, puertos, rutas del Gateway, perfiles y variables relevantes.

- Organización clara de perfiles (por ejemplo, dev, prod o equivalentes) cuando aplique.
- Separación adecuada de configuraciones sensibles o dependientes del entorno.
- **Despliegue local y/o remoto de microservicios**
 - Configuración de ejecución de los servicios de forma local (ya sea desde el IDE o mediante Docker).
 - Despliegue en al menos un entorno remoto (ej.: Railway, Render u otra plataforma) cuando sea parte del alcance definido por el docente.
 - Verificación de que los servicios se encuentren operativos y accesibles durante la defensa.
- **Buenas prácticas y calidad de código**
 - Código comentado cuando sea necesario.
 - Nombres de clases, métodos y variables significativos.
 - Estructura limpia y modular del proyecto.
 - Uso adecuado de paquetes según responsabilidad.
 - Eliminación de código muerto o duplicado antes de la entrega.
 - Estructura de proyecto limpia y coherente con el patrón CSR.
 - Nombres significativos para clases, métodos y variables.
 - Código comentado sólo cuando sea necesario para aclarar lógica compleja.
 - Eliminación de código muerto o duplicado antes de la entrega.
- **Integración de herramientas de trabajo colaborativo y gestión de versiones como Git**

Forma de entrega:

- El proyecto debe estar subido a un repositorio de GitHub público con acceso otorgado al docente y a los miembros del equipo.
- El repositorio debe contener:
 - Carpeta del proyecto o proyectos de microservicios.
 - Archivo README.md con:
 - Descripción del contexto o dominio del proyecto.
 - Nombres de los/las estudiantes.
 - Listado de microservicios implementados.

- Rutas principales del Gateway (cuando aplique).
- Enlaces a la documentación Swagger (local o remota).
- Instrucciones básicas de ejecución local y remota
- Pruebas unitarias con al menos el 80% de cobertura.
- Commits bien diseñados, distribuidos y generados que expliquen de manera correcta cada uno de ellos (no usar textos o descripciones no técnicas).
- **Si se detectan cambios así sean mínimos luego de la fecha de entrega en el repositorio por cualquiera de los integrantes del equipo y antes de su evaluación se les asignará un 1.0 de manera automática en la nota tanto grupal como individual.**
- Se subirá al **enlace de AVA grupal el código fuente desarrollado** (con los mismos elementos que se suben al repositorio) por un solo miembro del equipo. En caso de no entregar en AVA no se podrá realizar por ningún otro medio ni después de la fecha límite y se le generará un 1.0 tanto en la nota grupal como individual de manera automática.
- Se activará el enlace en **AVA individual escribiendo su nombre y apellido y número de equipo asignado**, así como el nombre de su aplicación o contexto. En caso de no activarlo en el plazo establecido no tendrá derecho a la defensa y se le asignará un 1.0 como calificación individual de manera automática (Si podrá acceder a la nota grupal en caso de haberlo activado)

Consideraciones para todos los estudiantes:

- La defensa técnica es individual.
- No se permite el uso de inteligencia artificial, herramientas automáticas de código o asistencia externa durante la sesión.
- El uso de internet debe limitarse estrictamente a la descarga de dependencias del proyecto Spring Boot.
- El/la docente monitoreará continuamente el trabajo del estudiante mediante observación directa o herramientas de control de laboratorio.
- Cualquier indicio de colaboración no autorizada o plagio implicará la anulación inmediata de la evaluación.
- Puedes solicitar repetir la pregunta si no la entiendes.
- Puedes anotar los pasos que vas a seguir.
- El/la docente evaluará tanto lo que haces como lo que explicas.

- Si algún recurso de la entrega del código no está implementado en la aplicación desarrollada, todos los ítems tanto grupales como individuales asociados a dicho elemento se calificarán de manera inmediata con un 0, esto incluye a los elementos donde se deben desarrollar o corregir errores en tiempo real durante la defensa.

3. Pauta de Evaluación Rúbrica

Categoría	% logro	Descripción niveles de logro
Muy buen desempeño	100%	Demuestra un desempeño destacado, evidenciando el logro de todos los aspectos evaluados en el indicador.
Buen desempeño	80%	Demuestra un alto desempeño del indicador, presentando pequeñas omisiones, dificultades y/o errores.
Desempeño aceptable	60%	Demuestra un desempeño competente, evidenciando el logro de los elementos básicos del indicador, pero con omisiones, dificultades o errores.
Desempeño incipiente	30%	Presenta importantes omisiones, dificultades o errores en el desempeño, que no permiten evidenciar los elementos básicos del logro del indicador, por lo que no puede ser considerado competente.
Desempeño no logrado	0%	Presenta ausencia o incorrecto desempeño.

Indicador de Evaluación	Categorías de Respuesta				Ponderación Indicador de Evaluación
	Muy buen desempeño 100%	Desempeño aceptable 60%	Desempeño incipiente 30%	Desempeño no logrado 0%	
Dimensión Entrega de Encargo (Grupal)					

Subdirección de Diseño Instruccional

IE 1.1.1 Diseña endpoints REST aplicando principios de arquitectura distribuida, asignando métodos HTTP y nombres semánticos según la responsabilidad del servicio, y valida la comunicación y respuesta del microservicio conforme a los principios REST, asegurando la correcta interpretación de códigos y manejo de mensajes.	Diseña y estructura todos los endpoints REST con métodos HTTP correctos, rutas semánticas y organización coherente en los microservicios. Valida completamente las respuestas en Postman, emplea códigos HTTP apropiados y mantiene consistencia en formato JSON y manejo de mensajes, reflejando dominio de los principios REST.		Define endpoints funcionales y coherentes, con métodos HTTP correctos en la mayoría de los microservicios. Presenta algunas inconsistencias menores en rutas, nombres o códigos HTTP, pero la comunicación REST general funciona adecuadamente.	Los endpoints presentan problemas de definición, rutas poco claras o uso incorrecto de métodos HTTP. Existen inconsistencias visibles en la comunicación REST, respuestas incompletas o códigos HTTP mal aplicados.	Los endpoints están mal definidos, no siguen principios REST o no funcionan. No se logra comunicación adecuada en los microservicios.	4%
IE 1.2.1 Estructura el microservicio aplicando el patrón CSR (Controller–Service–Repository/Model), con separación real de responsabilidades y paquetes por capa.	Estructura correctamente los microservicios siguiendo el patrón CSR, con capas bien separadas, paquetes ordenados y responsabilidades claras. No hay mezcla de lógica entre capas y la estructura facilita la mantenibilidad del proyecto.		Aplica el patrón CSR en la mayoría de los microservicios, aunque se observan algunas inconsistencias menores en organización de paquetes o distribución de responsabilidades.	La estructura presenta confusión entre capas, con mezcla de responsabilidades o paquetes desordenados. CSR solo se aplica parcialmente.	No aplica el patrón CSR o la estructura del proyecto es incorrecta, afectando la funcionalidad de los microservicios.	2%
IE 1.3.1 Implementa mecanismos de validación de datos de entrada, asegurando la integridad de la información procesada por el microservicio.	Implementa validaciones completas y consistentes en todos los puntos críticos de entrada de datos, cubriendo todos los microservicios. Los endpoints rechazan correctamente		Las validaciones están implementadas en la mayoría de los microservicios, con algunos casos no cubiertos o mensajes menos claros.	Las validaciones son básicas, incompletas o inconsistentes. Se aceptan datos inválidos en varios endpoints.	No implementa validaciones o estas no tienen efecto en la API.	3%

	solicitudes inválidas con mensajes y códigos adecuados.					
IE 1.3.2 Aplica estructuras de manejo de para controlar errores y mantener la estabilidad del servicio.	Controla errores globales y específicos con claridad y consistencia, devolviendo mensajes adecuados y códigos HTTP correctos en todos los microservicios. La API mantiene estabilidad incluso frente a fallas.		El manejo de errores está correctamente configurado en la mayoría de los microservicios, aunque con algunas inconsistencias menores en mensajes o códigos.	Presenta manejo de errores limitado o inconsistente. Algunos endpoints no gestionan fallas adecuadamente.	No implementa manejo de errores o las respuestas generadas son incorrectas.	3%
IE 2.1.1 Modela esquemas de base de datos normalizados y coherentes con el dominio, definiendo entidades JPA, atributos y relaciones según principios de diseño relacional y desacoplamiento, e integra operaciones CRUD completas conectando repositorios JpaRepository con la capa de servicio y los controladores REST, garantizando retornos JSON consistentes, validaciones aplicadas y funcionamiento estable del flujo interno del microservicio.	Modela entidades JPA normalizadas y coherentes en todos los microservicios, con relaciones correctas y atributos consistentes. Implementa CRUD estable, validado y funcional, con retornos JSON uniformes y lógica correctamente ordenada en capas.		Modela correctamente la mayoría de las entidades y CRUD funciona en la mayor parte de los microservicios, con fallas menores en relaciones, validaciones o retornos.	El modelado presenta errores, relaciones incorrectas o CRUD incompleto. Varias operaciones fallan o devuelven respuestas inconsistentes.	No modela adecuadamente el dominio y/o no implementa CRUD funcional.	4%
IE 2.2.1 Implementa reglas de negocio y	Implementa reglas de negocio claras,		Aplica reglas correctas en la mayoría de los	Reglas incompletas, mal aplicadas o poco claras.	El microservicio no contiene reglas de	4%

validaciones con Bean Validation en la capa de servicio, asegurando integridad de datos, coherencia del dominio y cumplimiento de restricciones funcionales definidas para el contexto del proyecto.	coherentes y alineadas al dominio, aplicando Bean Validation y lógica de servicio consistente en todos los microservicios.		microservicios, con algunas omisiones o inconsistencias menores.	El comportamiento esperado del negocio no se cumple en varios casos.	negocio o estas no funcionan.	
IE 2.3.1 Implementa manejo global y específico de excepciones, integrando logs estructurados para trazabilidad del sistema, garantizando respuestas claras, códigos HTTP adecuados y registro de eventos que permitan analizar errores en tiempo de ejecución.	Implementa manejo global y específico de excepciones en todos los microservicios, usando logs claros que permiten rastreo completo. Los códigos HTTP y mensajes son consistentes.		El manejo de excepciones funciona en la mayoría de los microservicios, aunque con detalles menores en logs o códigos HTTP.	Existen excepciones no controladas, logs insuficientes o respuestas incoherentes.	No implementa manejo de excepciones o logs funcionales.	3%
IE 2.4.1 Implementa comunicación REST entre microservicios mediante consumo o exposición de datos remotos, utilizando DTOs, controlando errores remotos y asegurando interoperabilidad completa en los flujos distribuidos del sistema, respetando	Implementa comunicación REST entre microservicios de forma estable, con DTOs adecuados, manejo de errores remotos y flujos distribuidos coherentes.		La comunicación remota funciona, pero presenta inconsistencias menores en rutas, DTOs o manejo de errores.	Existen errores frecuentes en la comunicación entre microservicios o incoherencias en el flujo de datos.	No implementa comunicación remota o esta falla completamente.	3%

convenciones REST y coherencia de datos.						
IE 2.5.1 Gestiona un repositorio GitHub ordenado, con commits técnicos, progresivos y distribuidos equitativamente entre los integrantes, y organiza tareas del proyecto en Trello u otra herramienta colaborativa, evidenciando roles y avance del equipo.	El repositorio está ordenado, con commits progresivos, técnicos y bien distribuidos. El tablero Trello muestra tareas claras, responsables y evidencia completa del avance del equipo.		El repositorio está mayormente ordenado y el tablero refleja parte del trabajo colaborativo con algunas inconsistencias menores.	El repositorio presenta desorden, commits agrupados o poco claros. Trello no refleja adecuadamente el flujo del trabajo.	No hay evidencia ordenada en GitHub ni gestión colaborativa en Trello.	2%
IE 3.1.1 Desarrolla pruebas unitarias para validar la lógica del microservicio, aplicando correctamente frameworks de testing y asegurando cobertura mínima sobre funciones y reglas de negocio.	Desarrolla pruebas unitarias claras, correctas y alineadas a las reglas de negocio. Usa mocks, asserts y estructuras de test adecuadamente, logrando una cobertura mínima del 80% demostrable.		Las pruebas unitarias funcionan y cubren los casos principales, aunque con estructura o cobertura parcial.	Las pruebas son insuficientes, incompletas o no cubren la lógica central.	No desarrolla pruebas unitarias funcionales.	4%
IE 3.2.1 Elabora documentación técnica clara, estructurada y trazable utilizando Swagger/OpenAPI, describiendo modelos, parámetros, rutas, respuestas, ejemplos funcionales y comportamiento real del microservicio.	Elabora documentación completa y profesional con Swagger/OpenAPI, incluyendo modelos, parámetros, rutas, ejemplos, respuestas y comportamiento real.		La documentación cubre la mayoría de los elementos requeridos, con algunos detalles pendientes.	Documentación incompleta, genérica o inconsistente con el código.	No documenta los microservicios o Swagger no funciona.	4%

IE 3.3.1 Configura correctamente los archivos YAML, propiedades de entorno, rutas del API Gateway y perfiles, y ejecuta el despliegue del ecosistema de microservicios en entornos locales o remotos (Docker, Railway, Render u otros), validando puertos, conexiones, disponibilidad y logs de ejecución, y describiendo el funcionamiento final del sistema distribuido.	Configura correctamente YAML, properties, perfiles y rutas de Gateway. Ejecuta el despliegue local/remoto validando puertos, conexiones, disponibilidad y logs. Todo el ecosistema queda operativo y estable.		Configura y despliega con algunos ajustes menores necesarios, pero el ecosistema funciona en general.	Existen errores recurrentes en configuración, puertos o despliegue que afectan la operatividad.	No logra configurar o desplegar el ecosistema de microservicios.	4%
Porcentaje Entrega de Encargo						40%
Dimensión Defensa						
IE 1.3.3 Ejecuta pruebas de comunicación REST para verificar respuestas y códigos HTTP de cada endpoint.	Ejecuta pruebas REST de manera precisa, validando correctamente respuestas, estructuras JSON y códigos HTTP en múltiples endpoints. Identifica comportamientos esperados y anómalos, demostrando comprensión completa del funcionamiento del microservicio.		Realiza pruebas REST de forma adecuada en la mayoría de los endpoints, con algunas omisiones o interpretaciones parciales de los resultados.	Ejecuta pruebas poco completas o con errores de interpretación. Le cuesta identificar fallas o validar correctamente códigos de estado.	No ejecuta pruebas REST, no comprende los resultados o no logra verificar respuestas y códigos HTTP.	5%
IE 2.1.3 Explica con claridad el modelado	Explica con claridad el diseño de entidades,		Explica correctamente la mayor parte del	La explicación es superficial, confusa o	No puede explicar el modelado,	5%

de entidades, relaciones y estructura del microservicio, justificando decisiones técnicas, normalización, consistencia del dominio y coherencia entre capas del patrón CSR.	atributos y relaciones, justificando cada decisión técnica. Demuestra comprensión profunda de normalización, consistencia del dominio y coherencia entre capas del patrón CSR.		modelado, aunque con justificaciones parciales o puntos incompletos.	demuestra comprensión limitada de relaciones, atributos o normalización.	desconoce relaciones, atributos o decisiones aplicadas.	
IE 2.1.4 Realiza modificaciones en vivo sobre CRUD, reglas de negocio o validaciones, asegurando compilación, funcionamiento correcto en pruebas REST y coherencia del flujo interno del microservicio.	Realiza modificaciones solicitadas en CRUD, reglas o validaciones de manera rápida y correcta. Compila sin errores, ejecuta pruebas REST y valida el funcionamiento de la API demostrando dominio pleno del flujo interno.		Realiza la modificación con algunos ajustes o correcciones menores, pero logra que la API funcione correctamente.	Realiza cambios incompletos, genera errores o requiere múltiples intentos para conseguir una ejecución parcial.	No logra realizar la modificación solicitada o deja el microservicio inoperativo.	10%
IE 2.2.2 Justifica la lógica de negocio implementada en la capa de servicio, demostrando comprensión del flujo interno del microservicio, mediante cambios en el código (añadiendo o ajustando validaciones o reglas), verificando su efecto en la API mediante pruebas REST.	Explica detalladamente la lógica de negocio implementada y realiza modificaciones solicitadas (validaciones o reglas), verificando su efecto mediante pruebas REST. Justifica de forma clara el impacto de los cambios.		Explica la lógica de negocio de forma correcta y logra realizar la modificación con pequeñas dificultades, validando su funcionamiento.	Tiene dificultades para explicar la lógica o realiza cambios que no tienen el efecto esperado.	No comprende la lógica de negocio o no puede modificar ni validar la funcionalidad solicitada.	4%

IE 2.3.2 Realiza cambios en el código demostrando control adecuado de logs, excepciones y códigos HTTP.	Identifica el origen de errores mediante logs y excepciones, y realiza cambios en el código para corregirlos, ajustando mensajes, códigos HTTP o flujos. Explica claramente el impacto del ajuste.		Reconoce el error y realiza ajustes correctos, aunque con explicaciones parciales o incompletas.	Identifica parcialmente el error o aplica una solución incompleta o incorrecta.	No identifica errores, no comprende logs o no realiza ajustes funcionales.	4%
IE 2.4.2 Explica el flujo de comunicación entre microservicios, demostrando la correcta obtención y uso de información remota, justificando su manejo y su impacto en el flujo del negocio, y realiza cambios en la comunicación remota comprobando la correcta obtención y uso de la información.	Explica claramente el flujo de comunicación entre microservicios, justificando rutas, DTOs y uso de datos remotos. Realiza cambios solicitados y valida correctamente la obtención y uso de información distribuida.		Explica adecuadamente el flujo y realiza la modificación con uno o dos ajustes menores necesarios para que funcione.	La explicación del flujo es incompleta y los cambios realizados no funcionan correctamente.	No explica la comunicación remota ni logra realizar ajustes funcionales.	4%
IE 2.5.2 Explica su aporte personal al proyecto, justificando commits, tareas asignadas y participación individual en el desarrollo del proyecto semestral, respaldando su relato con evidencia en las herramientas colaborativas utilizadas.	Presenta evidencia clara y verificable de sus aportes personales (commits, tareas, avances). Explica con seguridad qué desarrolló, cómo lo hizo y cuál fue su rol técnico dentro del equipo.		Entrega evidencia suficiente de participación, aunque con menor claridad o profundidad en su explicación.	La evidencia es débil, poco clara o no respalda completamente su participación.	No presenta evidencia verificable de aporte personal o no puede explicar su participación en el proyecto.	5%

IE 3.1.2 Explica con claridad el funcionamiento de las pruebas unitarias generadas en el proyecto, justificando decisiones técnicas y relacionándolas con el dominio del microservicio, e integra nuevas pruebas unitarias en tiempo acotado, aplicando correctamente la lógica requerida y manteniendo la consistencia del proyecto.	Explica con claridad las pruebas unitarias generadas, justificando mocks, asserts y casos evaluados. Integra en vivo nuevas pruebas funcionales en tiempo acotado, manteniendo consistencia del proyecto.		Explica adecuadamente las pruebas y logra integrar una nueva prueba con pequeños ajustes necesarios.	Explicación incompleta o integración fallida de nuevas pruebas.	No comprende las pruebas existentes ni puede agregar nuevas.	12%
IE 3.2.2 Explica la documentación técnica generada con Swagger/OpenAPI, recorriendo rutas, respuestas, modelos y ejemplos, y demostrando consistencia entre la documentación y el comportamiento real del código.	Explica Swagger/OpenAPI recorriendo rutas, modelos, parámetros y respuestas, demostrando que la documentación refleja fielmente el comportamiento real del microservicio.		Explica correctamente la mayoría de los elementos documentados, con pequeñas omisiones o inconsistencias.	Explicación superficial o inconsistencias relevantes entre documentación y código.	No puede explicar la documentación o esta no coincide con el funcionamiento real.	4%
IE 3.3.2 Realiza la configuración de manera individual y sin apoyo para ejecutar los microservicios en entornos locales y remotos de acuerdo a	Ejecuta sin apoyo todos los microservicios en entornos locales y/o remotos, ajustando perfiles, puertos y variables. Verifica disponibilidad, logs y		Levanta el ecosistema con algunos ajustes menores o indicaciones, validando su funcionamiento general.	Tiene dificultades para ejecutar los servicios o no valida completamente su funcionamiento.	No logra ejecutar los microservicios o depende totalmente de apoyo externo.	7%

lo que hayan construido como equipo.	conectividad entre servicios, demostrando autonomía técnica.					
Porcentaje Defensa						60%
Total						100%